GOSIM

GOSIM Shenzhen 2026

Mooncake Store：解耦架构中面向智能体强化学习的 KV 存储

GOSIM Shenzhen 2026



GOSIM

GOSIM Shenzhen 2026

01

Overall：项目与生态

参与单位、软硬件支持与社区影响



Mooncake 项目概况

大规模推理与强化学习中的传输、存储和通信组件



6.8k

GitHub Stars

源自 Kimi 推理系统

Apache 2.0
Transfer Engine / Store / EP / PG

1.3k

GitHub Forks

github.com/kvcache-ai/Mooncake

参与单位与社区

模型系统、云平台、芯片与开源框架共同参与

核心团队

阿里巴巴 / 清华大学 / 趋境科技 / 月之暗面

合作单位与社区

蚂蚁集团 ANT GROUP
HUAWEI
美团 Meituan
Tencent 腾讯
科大讯飞 iFLYTEK
NVIDIA
AMD
摩尔线程 MOORE THREADS
火山引擎
aws
intel

支持的硬件与云平台

GPU、NPU、异构加速器与云网络

NVIDIA

华为昇腾

AMD

寒武纪

摩尔线程

沐曦

海光

壁仞

平头哥

Sunrise

阿里云

AWS

设备内存与互连

运行时按设备和网络选择传输路径

设备 / 网络	传输路径	典型用途
NVIDIA CUDA	GPUDirect RDMA / NVLink / MNNVL	GPU KVCache 与权重传输
AMD / 海光	HIP / HSL	ROCm、DCU 设备通信
华为昇腾	Ascend / HCCL / UB	NPU 数据交换
MUSA / MACA / MLU / SUPA	对应运行时的设备内存适配	异构加速器传输
IB / RoCE / eRDMA / AWS EFA	RDMA / libfabric	跨节点高速网络
主机与存储	TCP / SHM / CXL / NVMe-oF	DRAM 共享与存储访问

推理软件生态

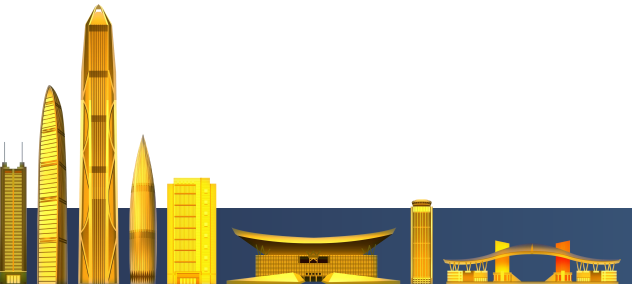
Mooncake 作为传输、缓存与通信后端

项目	Mooncake 组件	集成用途
vLLM / vLLM-Omni	TE、Store	PD 分离、共享 KV、多阶段数据交换
SGLang / SGLang-Omni	TE、Store、EP / PG	PD / EPD、HiCache、弹性 EP
NVIDIA Dynamo / NIXL	TE、Store 事件	传输插件、共享缓存路由
TensorRT-LLM / LMDeploy	TE、Store	KV 传输与共享存储
xLLM / RTP-LLM / vLLM-Ascend	TE 或 Store	异构部署、KV 池与缓存复用

训练与中间件生态

同一套对象存储承接多种数据

项目	数据对象	使用方式
Miles / Slime	Rollout、模型权重	对象后端与 P2P 权重更新
TransferQueue / NeMo RL / verl	训练样本、张量	异步数据队列与共享存储
TorchSpec	Hidden states	推理与草稿模型训练解耦
LMCache / FlexKV	KVCache	存储后端或 TE 传输后端
checkpoint-engine	模型权重	P2P checkpoint 分发



研究与工程影响

FAST 最佳论文、PyTorch 生态与跨框架集成



FAST 2025 最佳论文

KVCache-centric
Disaggregated Architecture

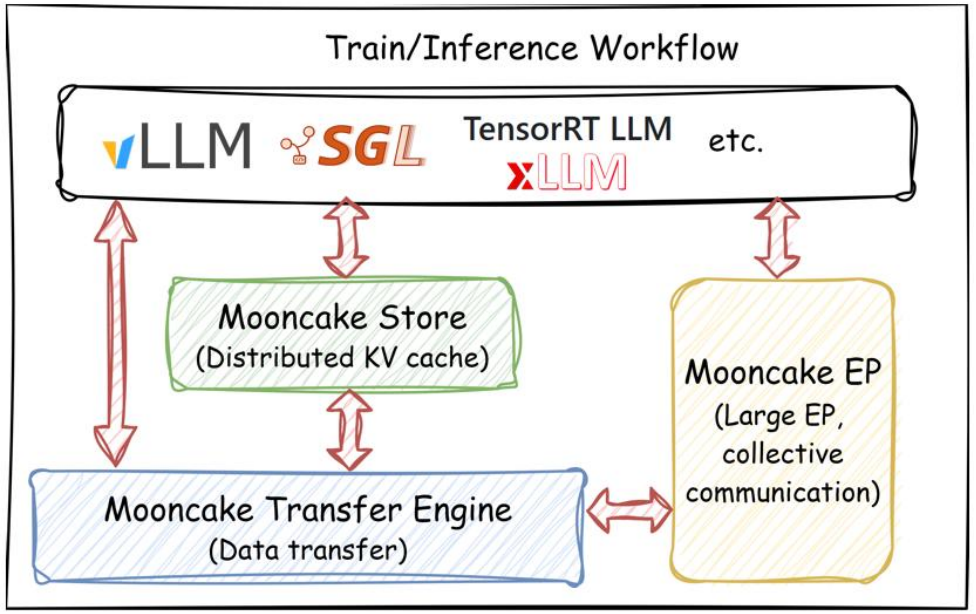
2024 Transfer Engine 开源

2025 Store 开源，接入 vLLM 与 SGLang

2026 加入 PyTorch Ecosystem

Mooncake 的基础组件

Transfer Engine 负责传输，Store 负责共享对象存储



Transfer Engine

批量数据传输
拓扑与多网卡管理

Mooncake Store

共享对象与元数据
DRAM / SSD 资源池

同一套基础设施承接 KVCache、rollout 数据和模型权重

计算与状态分别扩展

推理与训练框架决定计算，Mooncake 承接数据

KVCache	Prefill / Decode 间传输，多实例共享前缀
Rollout	生产者与消费者解耦，按阶段读取样本
模型权重	从训练布局转换为推理布局，发布新版本



GOSIM

GOSIM Shenzhen 2026

02

智能体与多轮上下文

缓存复用、跨实例访问与版本隔离



Agent 调用中的重复前缀

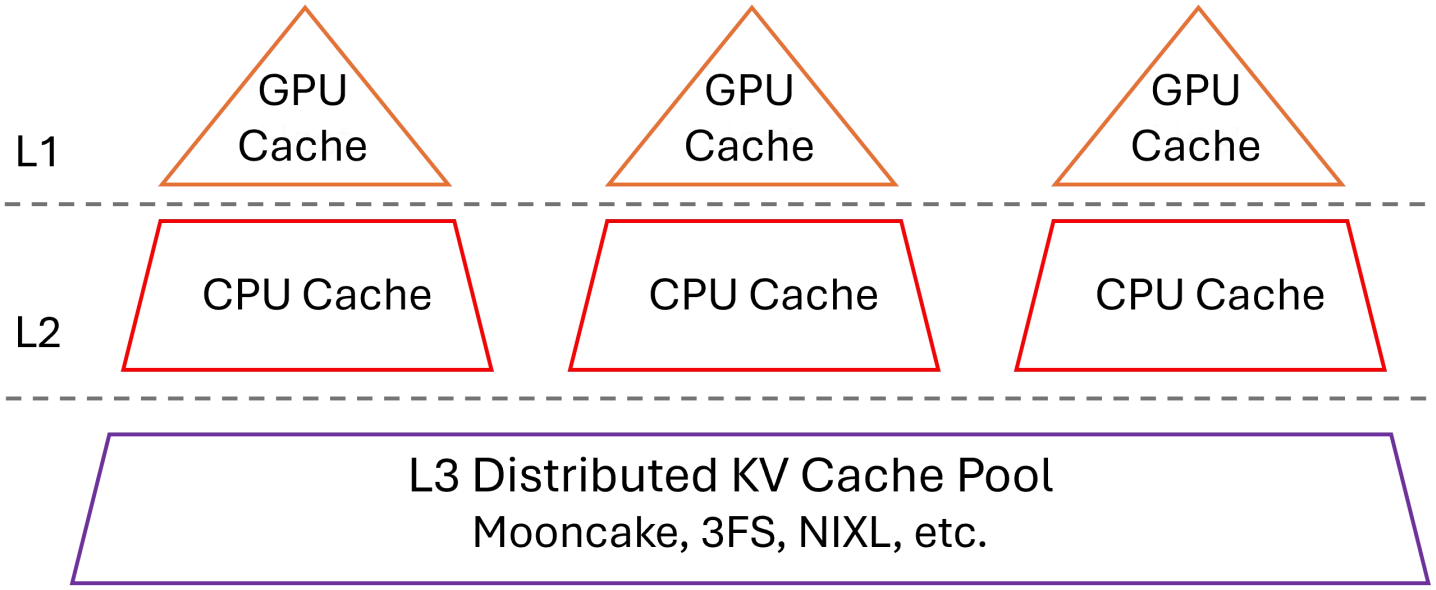
Codex / SWE-bench Pro 轨迹

长上下文中，大部分内容来自上一轮

Codex / SWE-bench Pro traces	观测值
样本量	610 条轨迹
轮数中位数	33 轮
第 30 轮上下文	约 80K tokens
平均输入 / 输出 token 比	131 : 1

Agentic RL 中的 KVCache 复用

多轮对话与工具调用包含大量重复前缀



共享前缀

多轮会话与并行轨迹
复用已经计算的上下文

分层存储

HBM / DRAM / SSD
按容量与带宽权衡

权重更新后，KVCache 需要按模型版本隔离或失效

跨实例复用需要共享 KV 池

容量 长会话占用持续增长，本机缓存容易被淘汰

调度 后续轮次可能落到另一个推理实例

版本 同一 token 前缀也必须对应兼容的模型状态



GOSIM

GOSIM Shenzhen 2026

03

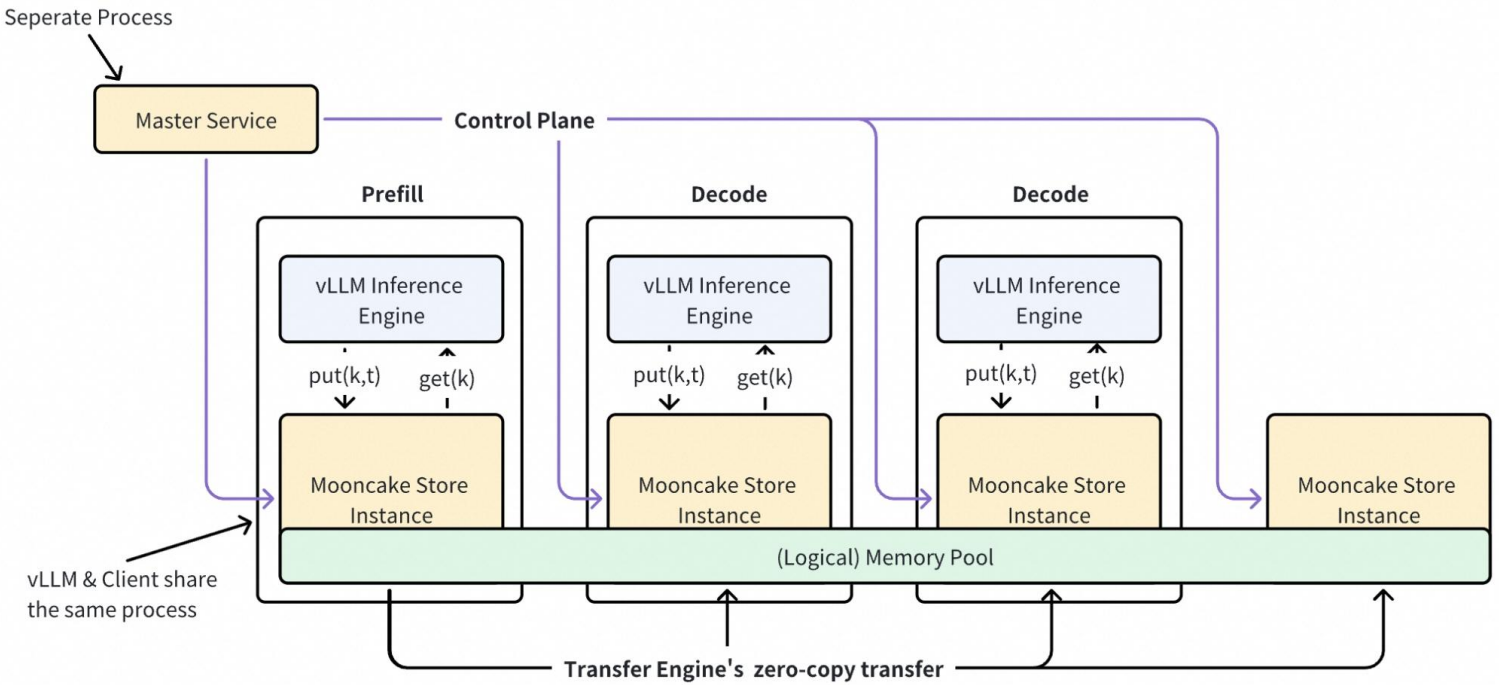
Store 的对象与分层存储

读写路径、副本、回收与恢复



Mooncake Store：共享对象与资源池

Master 管理元数据，Client 之间直接传输数据



计算实例独立扩缩容，共享对象由 Store 管理

Put：分配、传输与提交

大对象数据直接写入目标 Client

1 分配 Master 为对象选择 segment、空间与副本

2 传输 Transfer Engine 将 payload 写入目标位置

3 提交 传输成功后发布可读对象，失败路径释放预留



Get：查找副本并读入目标缓冲区

1 查询　　Master 返回可用副本与位置

2 读取　　优先选择合适路径，直接传入注册缓冲区

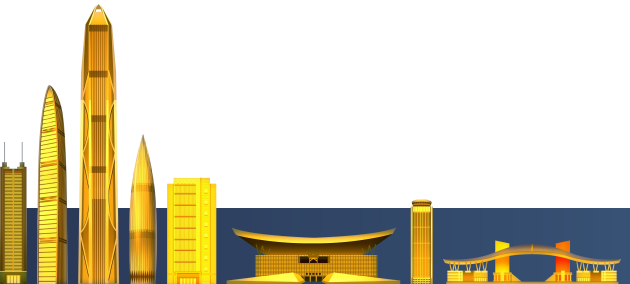
3 完成　　确认完成后，计算端再使用对应的数据



面向张量与批量访问的 API

按数据布局选择接口

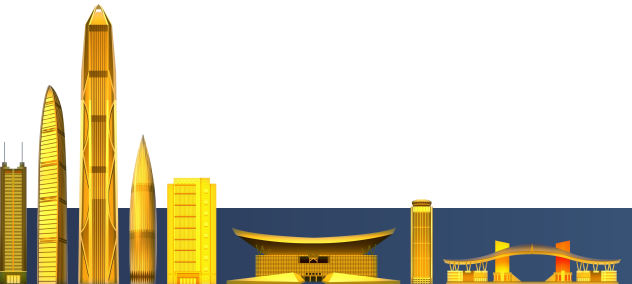
接口形式	适用场景
Put / Get	简单对象读写
Batch Put / Get	批量 KV blocks 与样本
From / Into buffer	使用调用者提供的注册缓冲区
Multi-parts / slices	非连续张量、按字段或切片访问



副本与对象放置

访问热点和缓存价值影响放置策略

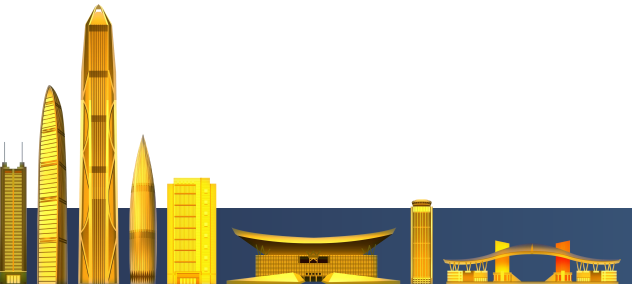
机制	作用
ReplicaConfig	指定副本数量与放置偏好
Preferred segment	倾向指定资源位置
Soft pin	提高重要对象的保留优先级
Hard pin	显式保护对象，配合释放与容量管理



对象生命周期与上层缓存策略

Store 与推理框架各自负责什么

环节	Store	推理 / 训练框架
对象标识	按 key 管理对象	定义前缀、模型版本与布局身份
有效读取	副本状态、租约与传输完成	确认数据可用于当前计算
淘汰与释放	内存分配、回收、pin 策略	决定保留哪些样本或前缀
重算与重试	返回命中、失败或缺失	选择回退计算与业务重试



分层缓存：HBM、DRAM 与 SSD

容量更大的层保存暂时离开 GPU 的状态

层级	主要角色	数据管理
GPU HBM	当前计算直接使用	推理引擎维护 KV slots
Host DRAM	低延迟换入与换出	HiCache / Connector 管理
Mooncake DRAM 池	跨实例共享、更大容量	Store 管理对象与副本
SSD / DFS	延长保留时间、降低容量成本	本地 offload 或 DFS 适配器

SSD Offload：写入、索引与回读

Master 维护对象的内存与磁盘副本位置

写入 客户端后台将待淘汰对象写入 SSD bucket

登记 写入完成后向 Master 登记磁盘副本

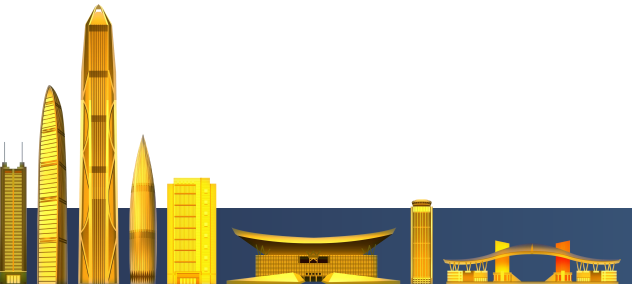
回读 持有 SSD 的 Client 读入缓冲区，再通过 TE 传输



DeepSeek 3FS：原生 USRBIO 访问

HF3FS 适配器处于实验阶段

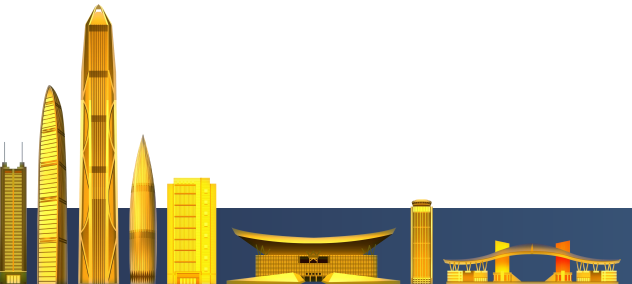
部分	实现方式
对象布局	Master 在共享 shard 文件中分配范围
数据访问	Client 使用 USRBIO 访问副本描述的区间
批量读取	并发读取多个对象，发挥存储并行性
配置要求	一致的 root、shard 数、容量与对齐方式



部署与恢复

推理实例、Store 资源与 Master 分别管理

部署方式	资源与恢复路径
嵌入式 Client	应用进程内调用，可同时贡献内存
独立 Store 服务	缓存资源与计算实例独立扩缩容
Master HA	etcd 选主、元数据恢复和状态追平
日志与快照	恢复对象索引、segment 与权重元数据



GOSIM

GOSIM Shenzhen 2026

04

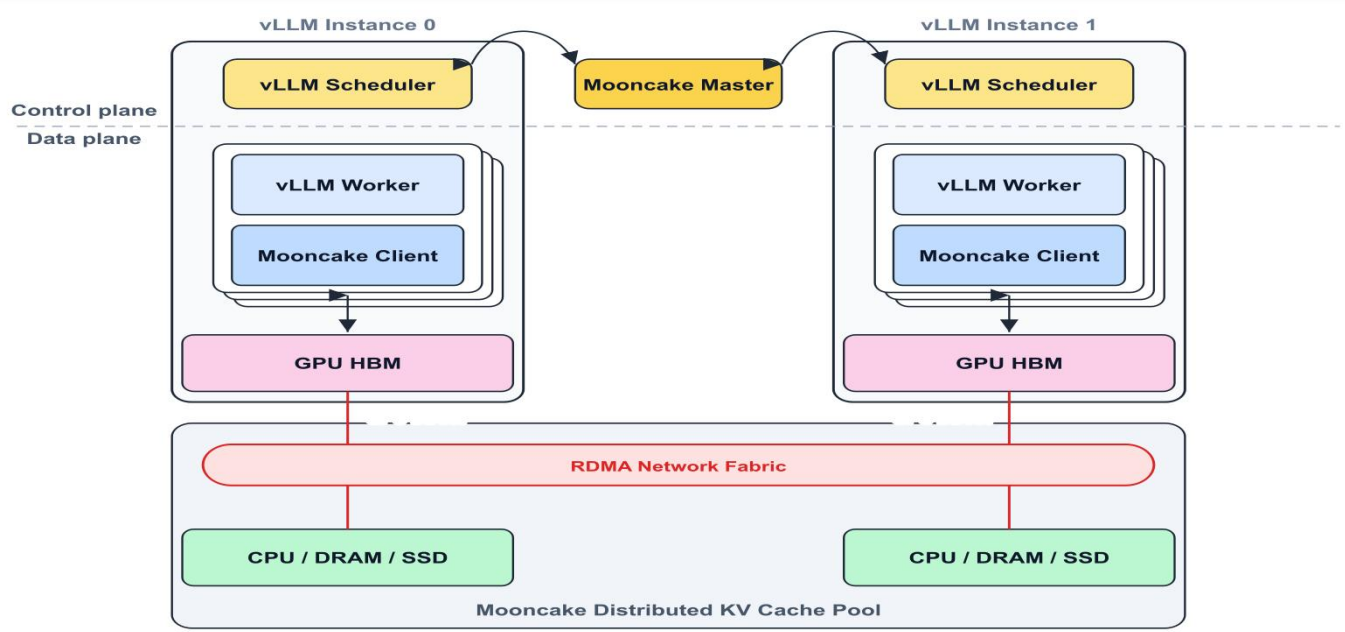
框架集成与缓存复用

vLLM、SGLang、Dynamo 和云原生部署



与 vLLM 的合作：共享 KVCache

MooncakeStoreConnector 接入 KVConnector 接口



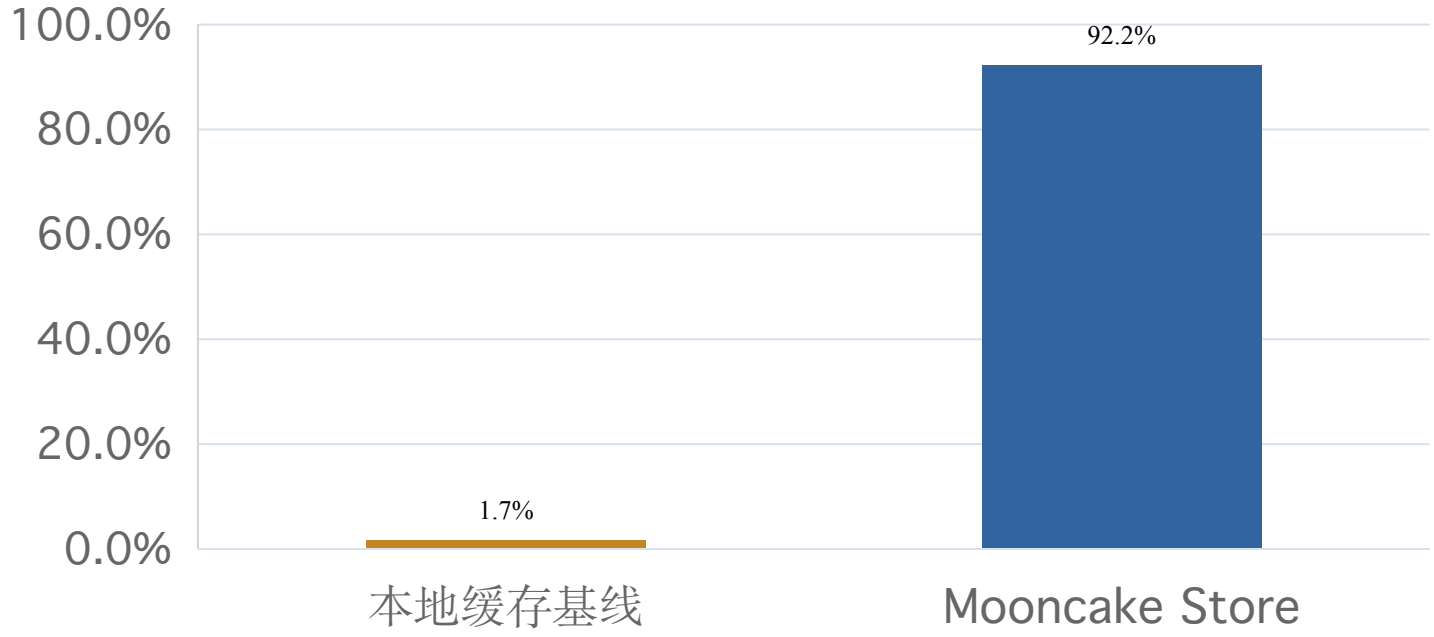
Scheduler
查询连续前缀与各 shard 命中

Worker
异步加载、写入与 RDMA 传输

MultiConnector
组合 PD 直传与共享 KV 池

vLLM Agentic trace 测试

Kimi-2.5 NVFP4， 1P1D， 12 × GB200



3.8 ×

吞吐提升

46 ×

P50 TTFT 加速

8.6 ×

端到端延迟加速

缓存命中率：1.7% 提升到 92.2%

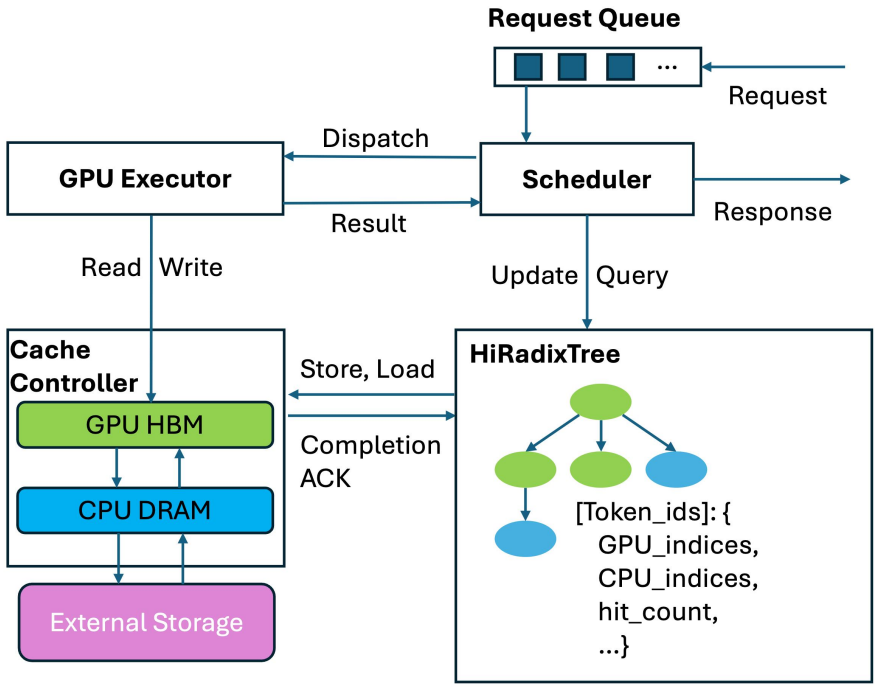
vLLM 扩容：12 到 60 张 GB200

独立的合成多轮扩容测试

设置	数值
上下文	20K 公共前缀，首轮输入 10K
后续轮次	每轮新增 2,048 输入、900 输出，共 30 轮
并发会话	75、150、225、300、375
调度与结果	Round-robin，命中率持续高于 95%

SGLang HiCache：GPU、Host 与共享存储

Mooncake 提供 external storage backend



HiRadixTree
记录前缀与分层缓存位置

Cache Controller
控制写入、预取与换入

Mooncake Store
提供跨实例共享对象池

HiCache 多轮测试：容量影响后续轮次

Qwen3-235B-A22B，8 × H800，输出长度设为 1

GPU only	GPU 容量限制可保留的历史前缀
Host L2	增加本机缓存后，早期轮次读取开销较低
Mooncake	上下文继续增长时，共享池维持更多前缀命中



Dynamo：根据缓存层级选择 worker

路由同时考虑本机缓存与共享池

信息来源	路由器获得的信息
Worker KV events	GPU 与 Host 各层的 stored / removed 事件
Mooncake object events	共享池中可取回的对象
候选 worker 评分	本地命中、远端取回与重算的成本
Dynamo + HiCache	将可取回的前缀计入调度判断

Mooncake Conductor：Python 缓存索引接口

通过 ConductorClient 订阅缓存事件、查询前缀命中

API / 数据流	用途
register() + KV events	注册发布者并持续接收缓存事件
query()	查询 prompt 在各实例的连续前缀命中
list / global view	查看服务与全局缓存索引状态
上层路由器	结合缓存局部性和负载选择服务实例

缓存事件持续更新索引，query() 查询当前前缀命中

与 NVIDIA Dynamo 的合作

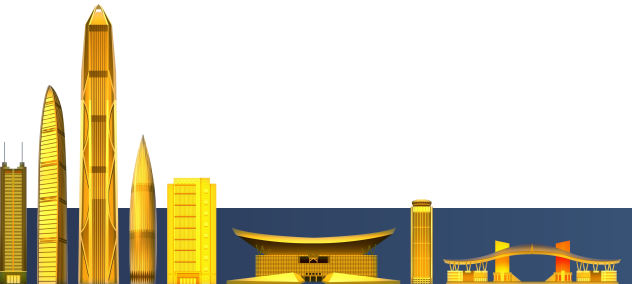
传输、缓存事件、路由与存储接口

合作方向	当前接口或进展
NIXL 传输插件	NIXL 可调用 Mooncake Transfer Engine
共享缓存路由	HiCache 分层事件与 Mooncake 对象事件
平台与工具	CUDA 13、MNNVL、Rust API、Mooncake traces
KVBM 的 G4 存储	Mooncake Store 后端处于开放 PR

DeepSeek 推理：PD 与专家并行

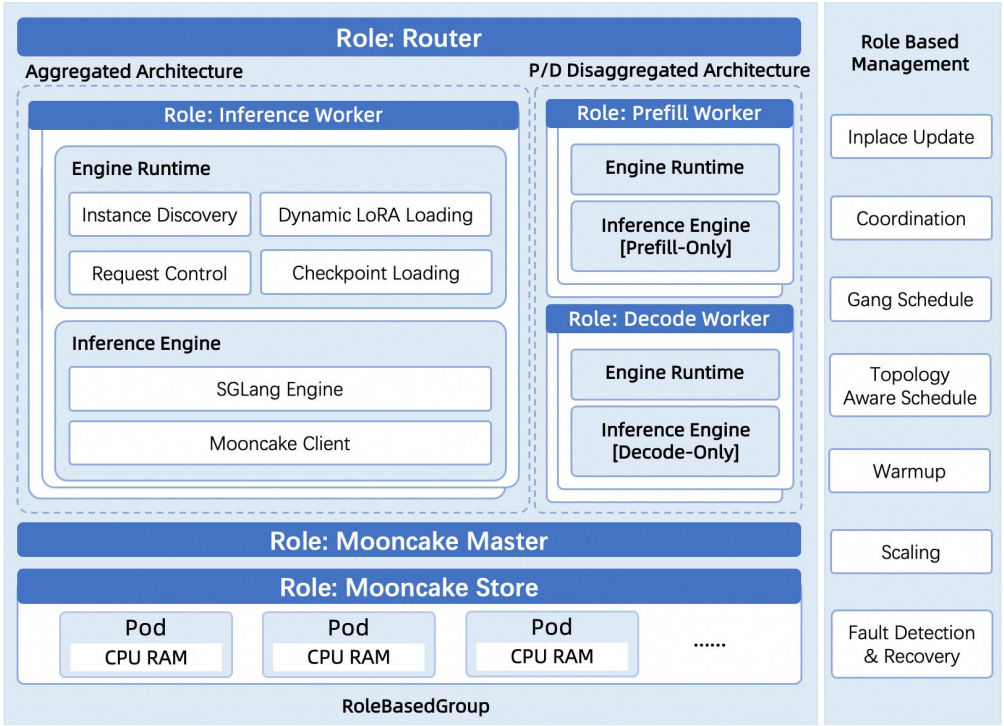
SGLang 的 PD 部署与 Elastic EP

部分	系统分工
Prefill / Decode	独立部署，使用 Mooncake 传输 KVCache
DeepEP / DeepGEMM	专家通信与 MoE 计算
EPLB / Two-batch overlap	专家负载均衡与计算通信重叠
Elastic EP	通过 Mooncake EP / PG 处理部分 rank 故障



生产部署：RBG、SGLang 与 Mooncake

按角色管理计算与共享存储



计算角色

Router、Prefill、Decode
按负载独立扩缩容

存储角色

Master 与 Store 独立部署
跨实例共享缓存

计算实例独立扩缩容，共享缓存由 Store 管理

GOSIM

GOSIM Shenzhen 2026

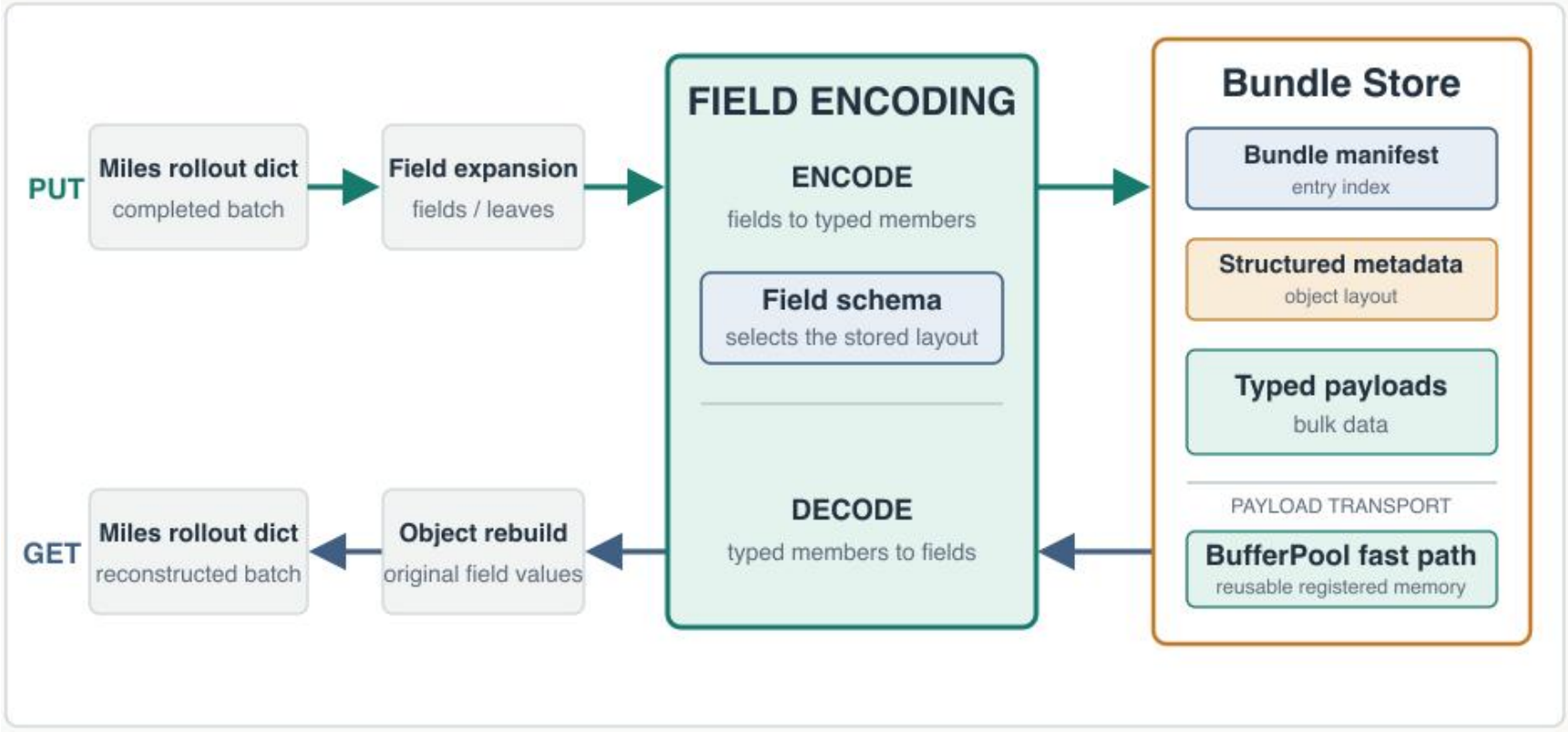
05

强化学习中的共享状态

样本、权重与 KVCache 的不同生命周期



结构化对象：元数据与 payload 分开组织



Schema 决定字段布局，manifest 记录对象组成，GET 恢复原始字段

Miles：替换 rollout 的 Ray Object Store 路径

调度与训练接口保持原有语义

步骤	交接内容	负责组件
发布	完整 rollout dict 存入 Store，返回引用	Mooncake backend
调度	传递轻量引用，决定 trainer 消费顺序	Miles / Ray
读取与清理	恢复同一份字典，消费后释放结果和对象	Trainer + Mooncake

--object-store-backend mooncake

NeMo RL：TransferQueue 数据面与可选 GDR

GPU tensor 直接读写共享主机内存池

位置	职责
Rollout / Trainer	生产或消费 GPU tensor
TransferQueue	元数据、字段可见性与消费调度
Mooncake GDR 路径	GPU 客户端与远端 host memory 直接传输
共享内存池	保存排队对象，管理生命周期与回收

GDR 省去客户端 CPU staging，数据仍存放在共享 host memory 中

Store 权重快照：保存后按目标布局读取

写入快照	begin_weight_snapshot write_tensor，再 commit
读取快照	load_manifest、plan_load、load 按目标 placement 选择 payload ranges
版本约束	固定 revision 与 weight generation 传输期间保持数据和内存有效

Store 保存快照，框架负责激活新权重与切流

模型更新后的状态管理

权重版本贯穿 rollout、缓存与训练

对象	需要保留的身份	更新或释放时机
KVCache	模型版本、前缀、dtype、布局	权重切换后隔离或失效
Rollout 样本	样本 ID、策略版本、字段与行	消费者完成后释放
权重快照	版本 ID、分片布局、manifest	新版本就绪后切换与回收

系统性能：命中、等待与有效吞吐

环节	观察指标
缓存	Token / byte 命中率，各层容量与淘汰
数据面	Put / Get 延迟、有效带宽、失败重试
推理	TTFT、TPOT、端到端 p95 / p99
强化学习	样本交接等待、权重更新、有效样本吞吐

Mooncake Store 的作用

共享容量	让 KVCache 跨实例保留与复用
数据交接	以对象承接样本、hidden states 和权重
生命周期	将计算扩缩容与缓存资源管理分开



GOSIM

GOSIM Shenzhen 2026

Thanks!

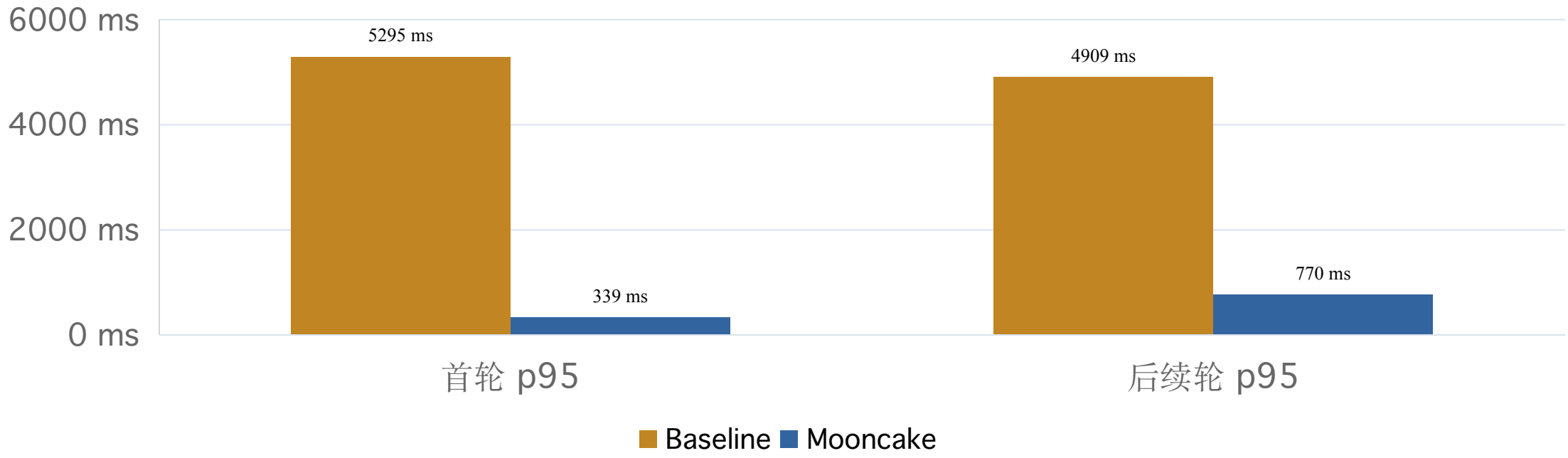
Q&A

github.com/kvcache-ai/Mooncake



OpenClaw 多会话案例

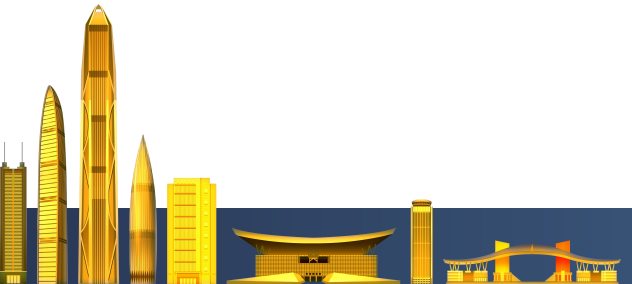
多会话请求的 p95 延迟（ms）



该案例中，后续轮 p50 为 171 ms / 169 ms

几种保留机制的区别

机制	保护的内容
Lease	读写期间的对象或缓冲区有效性
Soft / hard pin	对象回收策略
模型版本隔离	KVCache 与当前计算的语义兼容
HA 元数据快照	Store 对象索引与资源状态恢复



DeepSeek 生态：模型推理与 3FS

PD 分离、专家通信和分层缓存

方向	与 Mooncake 的结合
DeepSeek V3 / R1 推理	SGLang 使用 Mooncake 进行 PD 数据传输
DeepEP 与 Elastic EP	兼容专家通信接口，加入 rank 故障感知
DeepSeek 3FS	Mooncake Store 对接 HF3FS USRBIO
当前进展	PD / EP 已有案例，3FS 新适配器仍属实验性

代码、文档与技术资料

主题	入口
Store 架构与 API	kvcache-ai.github.io/Mooncake
vLLM 共享缓存	vllm-project.github.io/2026/05/06/mooncake-store.html
SGLang HiCache	Mooncake 文档：design/hicache-design
Dynamo 分层路由	Dynamo 文档：Using HiCache
3FS USRBIO	Mooncake 文档：3FS-USRBIO-Plugin